설비 고장을 예측하는 AI 데이터 분석

Seaborn 기초

분포와 범주 시각화

posco

CONTENTS

목차

01 Seaborn 시작과 분포 — histplot으로 분포 읽기

02 박스플롯과 범주형 비교 — 이상신호와 그룹 차이 읽기

posco

posco

01

Seaborn 시작과 분포

histplot으로 분포 읽기 — 시각화 도구 준비부터 hue 비교까지

왜 데이터를 그림으로 보는가

숫자 표로는 안 보이던 패턴이 그림에서는 한눈에 드러남

표 속 평균·표준편차만 보면 보이지 않는 신호
같은 데이터라도 그림 한 장에서 단번에 드러남

숫자 비교는 약하지만 모양·색 차이를 알아채는 능력은 강함
정상 상태의 모양을 알아야 이상 신호를 빨리 잡아냄
시각화는 꾸미는 작업이 아니라 신호를 빨리 찾는 분석 도구

posco

시각화가 필요한 이유

숫자엔 약하고 그림엔 강한 사람의 뇌

사람의 뇌는 숫자 비교엔 약하고, 그림의 모양·색 차이엔 강함

> 설비 한 대에서 1초마다 측정하면 하루 **8만 6천 개** 넘는 숫자가 쌓임
> 표·평균만으로는 안에 숨은 변화를 잡기 어려움

같은 데이터라도 그림으로 그리면 서서히 오르거나 갑자기 튀는 변화가 단번에 보임

왜 Seaborn을 쓰는가

적은 코드로 통계 그래프를 깔끔하게 그려 주는 도구

Matplotlib은 강력하지만 손이 많이 가는 도구

색 정하고 축 정하고 여러 줄을 써야 그래프 한 장이 완성
Seaborn은 이 불편함을 줄이려 만들어진 도구

Pandas 데이터프레임을 그대로 넣으면 색·디자인을 알아서 처리
양품·불량 색 구분 분포 겹쳐 그리기도 hue 한 단어면 끝

posco

Seaborn의 세 가지 장점

탐색 속도를 높이는 세 가지 강점

01 · 적은 코드

함수 하나로 통계 그래프 완성

함수 하나로 빠르게 그리기

코드가 짧을수록 데이터 탐색이 빨라짐

02 · Pandas 궁합

데이터프레임 그대로 입력

DataFrame을 그대로 시각화

앞서에서 배운 데이터프레임이 그대로 연결

03 · 통계 특화

분포·박스플롯·heatmap 기본 제공

분석에 필요한 그래프 기본 탑재

설비 데이터 분석에 필요한 그래프 대부분 포함

여러 그래프를 빠르게 그려보며 탐색하는 일에 최적

posco

Seaborn과 Matplotlib의 협업

sns 본체 → plt 제목·축 → plt.show 세 단계 흐름

STEP 1 · sns

그래프 그리기

그래프 본체는 **sns 함수**로 그리기

STEP 2 · plt

제목·라벨 달기

제목·축 이름은 **plt 함수**로 달기

STEP 3 · plt.show

화면에 띄우기

마지막 표시도 **plt.show**로 마침표

Seaborn을 쓴다고 Matplotlib을 안 쓰는 게 아님 — 둘은 한 팀

협업 구조 이해하기

sns는 본체, plt는 마무리 — 오늘 배우는 모든 그래프의 공통 패턴

Seaborn은 Matplotlib 위에서 동작

sns로 그린 그래프를 plt 함수로 이어 다듬을 수 있음

코드 맨 위에서 두 도구를 함께 불러오기

Seaborn 예제에 plt가 섞여 있어도 자연스럽게 읽힘

import와 set_theme — 환경 준비

작업 시작 시 맨 위에 두는 기본 패턴 — 도구 불러오기 + 테마 한 번 지정

CODE

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
```

set_theme는 그날 작업 맨 처음 딱 한 번만 실행 — 그래프마다 반복 불필요

posco

별명과 set_theme의 역할

pd · plt · sns는 전 세계 공통 약속, 임의로 바꾸지 않음

별명은 코드를 짧고 읽기 쉽게 만드는 장치

모두가 같은 약속을 쓰므로 임의로 바꾸지 않기

set_theme의 context="talk" — 발표용 큰 글자

뒷자리에서도 잘 보이도록 글자 크기 키우기

그런데 이대로 그리면 한글 제목이 네모로 깨지는 문제 발생 — 다음 슬라이드에서 해결

한글 폰트 깨짐 해결하기

한글이 네모로 깨질 때 둘 중 한 가지 방법 사용

CODE

```
plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False
# 또는
import koreanize_matplotlib
```

윈도우는 Malgun Gothic, 맥은 AppleGothic — 환경별 글꼴 이름이 다름

폰트 깨짐의 원리

컴퓨터가 고장난 게 아니라 그릴 글자 모양을 못 찾은 것뿐

기본 글꼴에는 영어·숫자 모양만 들어 있음

한글 모양이 없어 자리만 잡고 네모로 표시되는 현상

해결책 — 한글 모양이 든 글꼴을 쓰라고 지정

작업 시작 시 **한 번만** 설정하면 충분

윈도우 / 맥 글꼴 이름이 다르니 자기 환경에 맞는 것 메모해 두기

posco

실습 데이터 불러오기와 구조 확인

데이터를 열고 구조부터 확인 — shape · head · columns

CODE

```
df = pd.read_csv("equipment_sensor.csv",
 encoding="utf-8-sig")
print(df.shape)
df.head()
```

posco

데이터 구조 — 두 종류의 컬럼

어떤 그래프를 쓸지는 컬럼 유형에서 결정됨

01 · 숫자형 5개

분포로 확인하는 컬럼

온도 · 압력 · 진동
전류 · 습도

숫자로 된 센서값 — 히스토그램으로 분포 보기

02 · 범주형 2개

글자로 된 범주값

설비라인 A · B · C
판정 양품 · 불량

그룹 개수·평균은 countplot·barplot으로 보기

03 · 의도 포함

결측치와 이상치 포함

압력 9개
습도 18개 결측

앞서에서 배운 결측치를 여기서 다시 만나기

가장 먼저 배울 그래프는 분포 그래프 — 분포가 무엇인지부터 짚기

분포란 무엇인가

값들이 어느 구간에 얼마나 모여 있는지를 나타냄

일상에서 늘 느끼는 것이 분포
우리 반 키는 160~175 사이에 대부분 모여 있음

정상 상태의 분포를 알아야 이상 신호를 판단 가능
진동값이 보통 2 근처에 모인다는 걸 알아두면, 4를 기록했을 때 평소와 다른 주의 신호로 판단

분포는 무엇이 정상이고 무엇이 이상한지를 가르는 잣대

posco

분포가 정상 기준을 만든다

분포를 안다는 것 = 정상·이상의 잣대를 갖는 것

편의점 하루 음료 판매량도 보통 60~70개에 모임

극단값은 드묾 — 이것이 곧 분포의 모양

진동값이 평소 2 근처에 모인다는 걸 알면

4를 기록했을 때 **드문 값**이라 판단 가능

데이터 분석의 거의 모든 작업은 분포 보기에서 시작

posco

히스토그램의 원리

범위를 같은 너비로 쪼개고 개수를 세어 막대로 — 세 단계 흐름

STEP 1 · 구간 나누기

같은 너비 구간

값의 전체 범위를 같은 너비 구간으로 나누기

진동 1~6이면 1.0–1.5, 1.5–2.0 …

STEP 2 · 개수 세기

구간별 개수

각 구간에 든 측정값 개수 세기

각 칸에 들어간 값 카운트

STEP 3 · 막대 그리기

개수만큼 높이

센 개수만큼 막대를 높게 그리기

값이 많이 모인 구간은 막대가 큼

만들어지는 원리를 알면 그래프 모양도 정확히 읽힘

히스토그램 vs 막대그래프

입문자가 가장 많이 혼동하는 두 그래프 — 가로축 의미가 다름

구분	히스토그램	막대그래프
보는 것	숫자 하나의 분포	서로 떨어진 범주의 크기
가로축	연속 구간	독립 범주
막대 간격	붙어 있음	떨어져 있음

> 구간 개수는 **bins**라 부르며 곧 자세히 다룸

히스토그램은 숫자형 데이터 하나의 분포를 본다는 점, 분명히 기억

histplot 기본 사용법

data와 볼 컬럼 x만 지정하면 분포 완성

CODE

```
sns.histplot(data=df, x="온도")
plt.title("온도 분포")
plt.xlabel("온도(℃)")
plt.show()
```

x에 넣는 컬럼은 반드시 숫자형 — 설비라인 같은 범주형은 다른 그래프로

posco

histplot 마무리 3단계

앞으로 배울 박스플롯·산점도·heatmap에도 똑같이 적용되는 패턴

STEP 1 · 본체

sns.histplot

함수 이름만 바뀌고 본체 그리기는 동일

data · x 두 가지만 지정

STEP 2 · 라벨

plt.title · xlabel

제목과 축 이름 달기 — 단위까지 포함하면 가독성 ↑

예: 온도(℃), 진동(mm/s)

STEP 3 · 표시

plt.show

화면에 그래프 띄우기

모든 그래프의 공통 마침표

축 이름에 단위까지 넣으면 값의 크기를 정확히 이해 가능

posco

histplot 작성 시 주의점

가장 흔한 실수 — 따옴표 누락과 없는 컬럼 이름

컬럼 이름은 따옴표로 감싸기

데이터에 있는 이름과 정확히 일치시키기 — df.columns로 확인하는 습관

x에는 숫자형 컬럼만

숫자형이어야 분포가 의미를 가짐 · 범주형은 다른 그래프로

posco

실습 1. 첫 분포 그래프 그리기

histplot으로 한 센서의 분포를 그리고 축·제목 완성

목표

histplot으로 한 센서의 분포를 그리고 완성

단계

· data와 x에 볼 컬럼을 지정해 histplot 그리기

· 제목과 가로·세로축 이름 달기

· 값이 어디에 몰려 있는지 분포 모양 확인

예상 결과

전류값이 몰린 구간을 보여 주는 종 모양 분포

posco

실습 1. 첫 분포 그래프 그리기 정답

histplot으로 한 센서의 분포를 그리고 축·제목 완성

CODE · 실습 1

```
sns.histplot(data=df, x='전류')
plt.title('전류 분포'); plt.xlabel('전류(A)'); plt.ylabel('개수')
plt.show()
```

bins로 구간 수 조절하기

정답인 bins는 없음 — 분포가 가장 잘 보이는 값을 찾는 것이 요령

bins 값으로 막대 개수 정함

작게하면 뭉뚱그려지고 **크게**하면 자세해짐

300개 데이터는 보통 15~30 사이가 무난

정해진 값을 찾으려다 막히는 경우가 많음 — 직접 이리저리 바꿔보기

분포가 잘 보이는 값을 찾아 이리저리 바꿔보는 게 정상

너무 뭉뚱그려졌거나 너무 잘게 쪼개졌거나 — 직접 조절 가능

bins 값에 따른 변화

300개 데이터 기준 — 너무 작게 / 적절 / 너무 크게

너무 작게 · bins = 5

막대가 적어 분포가 뭉뚱그려짐

막대 다섯 개뿐이면 봉우리 위치만 대략 보임

적절 · bins = 20

봉우리와 꼬리가 모두 잘 보임

분포 모양이 가장 잘 드러남

너무 크게 · bins = 50

막대가 띄엄띄엄해짐

300개 데이터에 막대가 잘게 쪼개지면 빈 구간이 생김

bins 외에도 막대 위 부드러운 곡선 — kde 옵션이 유용

posco

kde 곡선으로 분포 모양 보기

kde=True로 막대 위에 분포 곡선 추가

CODE

```
sns.histplot(data=df, x="온도",
 bins=20, kde=True)
plt.title("온도 분포 (곡선 포함)")
plt.show()
```

곡선의 봉우리가 둘이면 서로 다른 두 집단이 섞였을 수 있음

kde 곡선 읽기와 주의점

곡선은 분포 전체 모양 — 정확한 개수는 막대로 확인

kde는 각 값 위에 작은 언덕을 올려 합친 것

봉우리 위치로 분포 흐름 파악

곡선은 추정 — 실제 없는 구간에도 이어질 수 있음

정확한 개수는 **막대**로 확인하기

곡선과 막대를 함께 보면 서로의 약점을 보완 가능

posco

실습 2. bins·kde 조절

bins로 구간 수를, kde로 분포 곡선을 조절

목표

bins로 구간 수를, kde로 분포 곡선을 조절

단계

· bins 값을 여러 개로 바꿔 가며 분포 그리기

· 구간 수에 따라 막대가 굵어지고 잘게 나뉘는 변화 관찰

· kde 곡선을 더해 분포 모양을 부드럽게 확인

예상 결과

bins 10·20·40에 따른 전류 분포 모양 차이

posco

실습 2. bins·kde 조절 정답

bins로 구간 수를, kde로 분포 곡선을 조절

CODE · 실습 2

```
for b in [10, 20, 40]:
    sns.histplot(data=df, x='전류', bins=b, kde=True)
    plt.title(f'전류 분포 bins={b}')
    plt.show()
```

posco

hue로 그룹별 분포 겹쳐 보기

hue에 범주 컬럼만 추가 → 그룹별 색 구분

CODE

```
sns.histplot(data=df, x="진동",
 hue="판정", kde=True)
plt.title("판정별 진동 분포")
plt.show()
```

hue 활용 포인트

hue에는 몇 가지로 딱 나뉘는 분류 컬럼만

hue에는 범주형 컬럼을 넣기

판정·설비라인은 적합 · 온도 같은 연속 숫자는 부적합(색이 수백 가지가 됨)

불량 분포가 양품보다 오른쪽에 치우치면

그 센서가 **고장 신호**일 단서

posco

분포 해석 — 세 가지 체크

중심 · 퍼짐 · 치우침 — 매번 체크리스트처럼 확인

01 · 중심

막대가 가장 높은 곳

값이 가장 많이 모인 위치

온도는 평균 약 74.8도 — 75도 근처가 가장 높음

02 · 퍼짐

좁게 vs 넓게

설비 안정성 신호

좁게 모임 = 일정 작동 · 넓음 = 들쭉날쭉

03 · 치우침

한쪽 꼬리가 김

드문 극단값 존재 신호

오른쪽 꼬리가 길면 설비 이상 신호 가능성

어떤 분포 그래프든 세 가지를 확인하면 핵심을 놓치지 않음

진동의 오른쪽 꼬리에 주목

중심·퍼짐·치우침 체크리스트로 핵심 신호 잡기

진동은 대부분 2 근처에 모임

오른쪽 끝 5 근처에 값 몇 개가 떨어져 있음

이 멀리 떨어진 값들이 다음 시간의 이상치

박스플롯에서 더 또렷하게 확인 가능

posco

실습 3. 양품·불량 분포 비교

hue로 판정별 분포를 색으로 겹쳐 비교

목표

hue로 판정별 분포를 색으로 겹쳐 비교

단계

· x에 온도, hue에 판정을 지정해 histplot 그리기

· kde를 더해 두 그룹의 분포 모양 비교

· 불량이 어느 쪽으로 치우쳤는지 확인

예상 결과

양품·불량 온도 분포가 색으로 겹쳐 표시

posco

실습 3. 양품·불량 분포 비교 정답

hue로 판정별 분포를 색으로 겹쳐 비교

CODE · 실습 3

```
sns.histplot(data=df, x='온도', hue='판정', kde=True)
plt.title('판정별 온도 분포'); plt.xlabel('온도(℃)')
plt.show()
```

실습 4. 여러 센서 분포 한눈에 보기

반복문으로 여러 센서 분포를 차례로 그리기

목표

반복문으로 여러 센서의 분포를 차례로 그려 비교

단계

· 컬럼과 단위 목록을 반복하며 각 센서 분포 그리기

· 각 그래프에 컬럼 이름과 단위로 제목·축 달기

· 센서마다 값이 몰린 정상 범위를 정리

예상 결과

온도·진동·전류 분포를 차례로 확인

posco

실습 4. 여러 센서 분포 한눈에 보기 정답

반복문으로 여러 센서 분포를 차례로 그리기

CODE · 실습 4

```
for col, unit in [('온도','℃'), ('진동','mm/s'), ('전류','A')]:
    sns.histplot(data=df, x=col, bins=20, kde=True)
    plt.title(f'{col} 분포'); plt.xlabel(f'{col}({unit})')
    plt.show()
```

posco

02

박스플롯과 범주형 비교

이상신호와 그룹 차이 읽기 — boxplot · countplot · barplot

박스플롯이 필요한 이유

분포 요약과 이상치 표시를 동시에 한눈에 보여줌

히스토그램의 약점 보완

정상은 상자 안 · 이상은 바깥 점이라는 직관적 구조

히스토그램은 분포는 보여줘도 정상 범위를 딱 잘라 말하기 애매
박스플롯은 분포 요약 + 이상치 표시 = 두 가지를 동시에 처리
히스토그램과는 역할이 분명히 다른 그래프

박스플롯의 강점

현장에서 몇 초 만에 이상 위치를 잡아내는 그래프

이상치를 자동으로 점으로 표시

진동의 오른쪽 꼬리가 상자 위 점으로 또렷해짐

여러 그룹을 나란히 비교 가능

어느 라인이 더 흔들리는지 즉시 비교

중앙값과 사분위수

상자만 보고도 데이터를 읽을 수 있는 통계적 의미

중앙값 — 크기순 줄 세웠을 때 정확히 가운데 오는 값

이상치 하나에 흔들리지 않는 안정적인 대표값

진동값 `2.0 2.1 2.2 2.3 8.0` 예시

평균 3.3 · 중앙값 2.2 — 더 정확한 대표값은 중앙값

이상치가 섞이기 쉬운 설비 데이터에서 더 믿을 만한 대표값

사분위수와 IQR

데이터를 4등분한 지점 — 25%(Q1) · 75%(Q3)

25% 지점이 Q1 · 75% 지점이 Q3

크기순으로 줄 세워 4등분한 경계 지점

진동 — Q1 약 2.06 · Q3 약 2.71

가운데 절반이 이 사이에 모임 — 이 범위가 **IQR**

앞서 이상치 처리에서 배운 IQR과 같은 개념

박스플롯의 네 부분

상자 · 중앙값선 · 수염 · 이상치점 — 한 장에서 분포 핵심 모두 읽기

01 · 상자

Q1 ~ Q3 범위

가운데 절반이 든 범위

길면 넓게 퍼짐 · 짧으면 좁게 모임

02 · 중앙값선

상자 안 가로선

데이터의 가운데 값

위에 치우치면 데이터가 위쪽에 몰림

03 · 수염

정상값의 끝

정상 범위의 경계선

수염 끝까지 정상으로 봄

04 · 이상치점

수염 밖 점

유독 크거나 작은 값

설비 점검에서 가장 먼저 봐야 할 점들

박스플롯 한 장에서 분포의 핵심을 모두 파악 가능

이상치 읽기와 진동 사례

앞서 IQR 방식과 동일 — 박스플롯이 자동 계산

이상치 기준은 앞서 IQR 방식과 동일

박스플롯이 자동으로 계산해 점으로 표시

진동 — 상한선 약 3.68을 넘는 값 6개

위쪽 점으로 찍힘 — **우선 점검 대상**

posco

boxplot 기본 사용법

단일 센서 분포는 y에 숫자 컬럼

CODE

```
sns.boxplot(data=df, y="진동")
plt.title("진동 박스플롯")
plt.ylabel("진동(mm/s)")
plt.show()
```

상자·수염·이상치 점이 모두 자동 — 사분위수를 직접 계산할 필요 없음

boxplot의 자동 계산

histplot과 같은 마무리 패턴 그대로

Q1·Q3 상자, 중앙값 선, 수염, 이상치 점까지 자동

함수 하나면 끝 — 직접 계산 불필요

마무리는 plt.title · plt.show로 동일

histplot은 x에 숫자, boxplot은 단일 분포 시 y에 숫자

두 함수 모두 숫자형 컬럼을 봄 — 입력 위치만 다름

boxplot 주의점

범주형은 단독 y가 아니라 그룹 나누는 용도로 사용

컬럼은 따옴표로 정확히 · 분포 볼 컬럼은 숫자형

설비라인·판정 같은 범주형을 y에 단독으로 넣으면 의미 있는 상자가 안 나옴

결측치는 자동으로 빼고 그림

남은 값만으로 상자 그림 — 오류 없음

여러 센서를 박스플롯으로 그려 직접 이상치 찾기 (다음 실습)

posco

실습 5. 단일 센서 박스플롯

boxplot으로 한 센서의 사분위수·이상치 확인

목표

boxplot으로 한 센서의 사분위수와 이상치 확인

단계

· y에 볼 센서를 지정해 박스플롯 그리기

· 상자(가운데 절반)와 수염, 이상치 점 위치 확인

· 제목·축 이름 붙이기

예상 결과

전류의 상자·수염과 위쪽 이상치 점

posco

실습 5. 단일 센서 박스플롯 정답

boxplot으로 한 센서의 사분위수·이상치 확인

CODE · 실습 5

```
sns.boxplot(data=df, y='전류')
plt.title('전류 박스플롯'); plt.ylabel('전류(A)')
plt.show()
```

범주별 박스플롯

x에 범주, y에 숫자 → 그룹별 상자

CODE

```
sns.boxplot(data=df, x="설비라인",
 y="진동")
plt.title("라인별 진동 분포")
plt.show()
```

histplot은 x에 숫자, 범주별 boxplot은 x에 범주 — 헷갈리지 않기

범주별 비교로 우선순위 정하기

나란히 비교 → 어디를 먼저 봐야 할지 우선순위 결정

세 라인 진동 중앙값은 약 2.32~2.42로 비슷

하지만 라인마다 위쪽 이상치 점이 따로 찍힘

특정 라인에만 이상치가 많다면 그 라인 우선 점검

그룹을 나란히 비교하면 우선순위가 그래프에서 드러남

posco

hue로 두 기준 동시 비교

hue 추가 → 라인별 × 판정별 이중 비교

CODE

```
sns.boxplot(data=df, x="설비라인",
 y="진동", hue="판정")
plt.title("라인별 · 판정별 진동")
plt.show()
```

각 라인마다 양품·불량 상자가 두 개씩 나란히

posco

일관된 패턴이 신뢰할 신호

한두 군데 패턴은 우연 · 모든 그룹 반복 패턴은 진짜 신호

가로축은 라인, 색은 양품·불량의 이중 비교

두 기준을 한 번에 비교할 수 있는 강력한 시각화

라인과 무관하게 반복되는 패턴 = 믿을 만한 고장 신호

단 hue까지 넣으면 상자가 많아져 복잡 — 기준이 분명할 때만 사용

실습 6. 범주별 박스플롯 비교

라인별, 그리고 라인×판정 교차로 분포 비교

목표

x에 범주를, hue에 판정을 더해 범주별 분포 비교

단계

· x에 설비라인을 지정해 라인별 상자를 나란히 그리기

· hue에 판정을 더해 라인마다 양품·불량 상자 비교

· 일관된 패턴이 신뢰할 신호임을 확인

예상 결과

라인별 온도 상자, 그리고 라인×판정 교차 비교

posco

실습 6. 범주별 박스플롯 비교 정답

라인별, 그리고 라인×판정 교차로 분포 비교

CODE · 실습 6

```
sns.boxplot(data=df, x='설비라인', y='온도')          # 라인별
plt.title('라인별 온도 분포'); plt.show()
sns.boxplot(data=df, x='설비라인', y='온도', hue='판정')  # 교차
plt.title('라인별·판정별 온도 비교'); plt.show()
```

posco

범주형 데이터를 세는 방법

숫자형은 분포로 · 범주형은 개수 또는 그룹별 값으로

A라인이 B라인보다 크다는 말은 어색
설비라인엔 분포라는 말이 어울리지 않음

숫자형 → 분포
범주형 → 개수(countplot) 또는 그룹별 평균(barplot)
어떤 그래프를 쓸지가 여기서 결정

posco

범주형을 보는 두 방식

범주별 개수를 보는 countplot · 범주별 평균을 보는 barplot

01 · 개수 보기

countplot

각 범주가 몇 개인지 세어 막대로

A라인 데이터 몇 개, B라인 몇 개를 막대로 그리기

02 · 평균 보기

barplot

범주별 숫자값의 평균을 막대로

라인별 평균 진동이 얼마인지 막대로 그리기

먼저 가장 단순한 것 — countplot부터

posco

countplot으로 개수 비교

x만 지정하면 개수를 자동 집계

CODE

```
sns.countplot(data=df, x="설비라인")
plt.title("라인별 데이터 개수")
plt.show()
```

countplot 활용

데이터 불균형 확인에 유용

y를 따로 줄 필요 없이 x에 범주만 넣으면 자동 집계

개수가 막대 높이로 보임 — 입력이 가장 간단한 그래프

양품 226 vs 불량 74 — 한쪽 치우침이 바로 보임

hue=판정을 더하면 라인 안 양품·불량 개수까지 색으로 구분

posco

barplot으로 평균 비교

x(범주)와 y(숫자) 모두 지정 → 평균 막대

CODE

```
sns.barplot(data=df, x="설비라인",
 y="진동")
plt.title("라인별 평균 진동")
plt.show()
```

평균 진동 A 2.42 · B 2.47 · C 2.54 — countplot은 x만, barplot은 x와 y

barplot의 오차선

막대 위 가는 세로선은 평균의 신뢰도 표시

막대 높이는 범주별 평균값

합계나 중앙값이 아닌 평균이 기본

막대 위 가는 세로선 — 오차 범위

길면 평균이 덜 확실 · 짧으면 평균이 꽤 믿을 만함

posco

countplot vs barplot 차이

y축이 필요한지로 구분 — y 있으면 barplot, x만이면 countplot

구분	countplot	barplot
보는 것	범주별 개수	범주별 평균
필요 값	x만	x와 y
막대 높이	개수	평균값

> **y축이 필요한가** — 가장 확실한 구분 기준

생김새가 같은 막대라 결과만으론 구분 불가 — 코드 쓸 때 의도 분명히

두 그래프 구분 요령

코드 쓰는 순간 의도를 분명히 — 질문이 다름

질문이 "몇 개인가"면 countplot

질문이 "평균이 얼마인가"면 barplot

생김새가 같은 막대 — 결과만으론 구분 불가

"라인별 진동" 막연한 생각으로 countplot 쓰면 진동값 무시·개수만 셈

색 바꾸기는 다음 — 최신 버전에서 주의할 점 있음

palette와 hue 함께 쓰는 법

색만 바꾸려면 hue=x컬럼 + legend=False

CODE

```
sns.countplot(data=df, x="설비라인",
 hue="설비라인",
 palette="Set2", legend=False)
plt.show()
```

색만 바꾸고 싶을 때의 패턴 — hue에 x와 같은 컬럼, legend=False

palette 변화의 배경

규칙이 바뀐 이유 — 모호함 제거

예전엔 palette만 단독으로 써도 됐으나

0.14에서 제거 예정이라 경고 발생

색을 나누려면 hue로 기준을 정하도록 규칙 변경

처음부터 새 방식으로 익히기

posco

색 나누기 두 경우

색만 바꾸기 vs 기준으로 색 나누기 — 범례 처리가 다름

01 · 색만 바꾸기

가로축에 라인 이름이 이미 있음

hue=x와 같은 컬럼
+ legend=False

범례는 꺼버리기 — 가로축이 이미 알려줌

02 · 기준 색 나누기

가로축은 라인, 색은 양품·불량

hue=다른 컬럼(판정)
+ 범례 유지

범례가 어떤 색이 뭔지 알려줌

posco

실습 7. countplot·barplot

개수는 countplot, 평균은 barplot으로 범주 비교

목표

개수는 countplot, 평균은 barplot으로 범주 비교

단계

· countplot으로 라인별 데이터 개수를 막대로 세기

· barplot으로 라인별 평균 전류를 막대로 비교

· 색은 hue와 palette로 지정(범례는 끄기)

예상 결과

라인별 개수와 라인별 평균 전류 막대

posco

실습 7. countplot·barplot 정답

개수는 countplot, 평균은 barplot으로 범주 비교

CODE · 실습 7

```
sns.countplot(data=df, x='설비라인', hue='설비라인', palette='Set2', legend=False)
plt.title('라인별 데이터 개수'); plt.show()
sns.barplot(data=df, x='설비라인', y='전류', hue='설비라인', palette='pastel', legend=False)
plt.title('라인별 평균 전류'); plt.show()
```

posco

실습 8. 라인별 불량 비율 비교

라인별 양품·불량 개수를 그래프와 표로 정리

목표

라인별 양품·불량 개수를 그래프와 표로 정리

단계

· countplot에 hue로 판정을 넣어 라인별 양품·불량 개수 비교

· groupby와 unstack으로 라인×판정 개수 표 만들기

· 불량이 상대적으로 많은 라인 확인

예상 결과

라인별 양품·불량 막대와 개수 표(A라인 불량 30 등)

posco

실습 8. 라인별 불량 비율 비교 정답

라인별 양품·불량 개수를 그래프와 표로 정리

CODE · 실습 8

```
sns.countplot(data=df, x='설비라인', hue='판정')
plt.title('라인별 양품·불량 개수'); plt.show()
표 = df.groupby(['설비라인', '판정'], observed=True).size().unstack(fill_value=0)
print(표)   # A라인 불량30·양품100, B 29·70, C 15·56
```